

FlashERP: Comprehensive Project Implementation Plan & Technical Specification

Version: 1.0 (Detailed Specification)

Date: August 2026

Status: Draft / Pending Review

Prepared For: Team Lead and Management

1. Executive Summary & Project State

The FlashERP platform is an innovative B2C e-commerce engine specifically designed for high-urgency Flash Deals. These deals are powered by a real-time inventory reservation system that guarantees stock availability during the checkout process and prevents overselling under extreme concurrent load.

1.1 Current State Analysis

A thorough codebase audit of the existing monorepo reveals that the frontend UI (`client/` directory, built with Next.js and React) is structurally mature. It encompasses high-fidelity components, robust client-side routing, and utilizes static mock data for both the Admin Dashboard and the public B2C Storefront. Conversely, the backend service (`server/` directory, built with NestJS) is currently an uninitialized boilerplate containing no business logic or database connections.

1.2 Objective

The primary objective of this implementation plan is to transition the application from a frontend prototype to a fully integrated, scalable, and production-ready system. This requires architecting a highly concurrent backend API, implementing a complex Redis-driven reservation engine, designing a normalized database schema, and wiring all React components to live server integrations to handle real-world transactions.

2. System Architecture & Infrastructure

To support the high-burst traffic characteristic of Flash Deals, the architecture is designed to be highly concurrent, stateless at the application layer, and extremely resilient.

2.1 Core Technology Stack

- **Node.js (v20 LTS):** Chosen for long-term support stability, native fetch API, and V8 engine performance improvements crucial for handling high-concurrency event loops during flash sales.
- **Frontend Framework:** Next.js (v15.x) & React (v19.x). Selected because the client codebase is already built on this modern stack. It utilizes React Server Components (RSC) for superior SEO and fast initial page loads required for high B2C conversion rates.
- **Backend Framework:** NestJS (v10.x). Selected for its strict TypeScript architectural patterns, robust dependency injection, and out-of-the-box support for modular domains and WebSocket microservices.
- **Relational Database:** PostgreSQL (v16.x). Chosen for its advanced JSONB support, logical replication, and significantly improved query planner which is vital for complex, heavily-indexed product catalog filtering.
- **ORM (Object-Relational Mapping):** Prisma (v5.x). Preferred for its strict end-to-end type-safety and rapid schema iteration. Version 5 specifically brings massive cold-start improvements and highly optimized JOIN queries compared to legacy versions.
- **In-Memory Datastore:** Redis (v7.2+). This is a critical infrastructure requirement. Version 7.2+ is mandated for its advanced KeySpace Notifications (`notify-keyspace-events EX`) and native Lua scripting capabilities, which are required to handle single-threaded atomic cart reservations and time-to-live (TTL) countdowns.
- **Frontend Integration:** Axios (v1.6+) will be used for secure, interceptor-driven REST API data fetching. React Query (v5.x) will manage client-side caching, pagination state, and optimistic UI updates to ensure a snappy user experience while minimizing redundant network calls.

2.2 High-Level Architectural Patterns

To survive extreme concurrency without bottlenecks, the codebase adheres to the following structural patterns:

- **Server - Modular Monolith:** The NestJS API is strictly divided into independent domains (Auth, Orders, Products). This allows rapid feature development while keeping the architecture primed for a future microservice split if necessary.
- **Server - Stateless Application Layer:** The Node.js instances hold zero state. All session data, rate limiting, and cart locks reside in Redis, allowing the infrastructure to scale horizontally (adding more containers) instantly under load.
- **Server - Event-Driven Background Processing:** Heavy operations (e.g., dispatching 10,000 SMS reminders or unlocking expired carts) are offloaded to background message queues (BullMQ) or Redis KeySpace listeners so the main HTTP event loop is never blocked.

- **Frontend - Hybrid Rendering (RSC):** The Next.js storefront utilizes React Server Components to pre-render HTML on the server, providing near-instant First Contentful Paint (FCP) and perfect SEO for product pages.
- **Frontend - Optimistic UI & WebSockets:** React Query immediately updates the UI during interactions (like adding to a wishlist) before the server responds, masking network latency. Live inventory drops are broadcasted via WebSockets (Socket.io) to prevent 10,000 clients from polling the REST API simultaneously.

2.3 Third-Party Integrations

- **Payment Gateway:** Razorpay, Stripe, or Cashfree. These gateways will process both nominal micro-transactions (Flash Deal Registration fees) and full cart order checkouts.
 - **Communications Platform:** AWS SES or SendGrid for all transactional emails (Invoices, Password Resets). Twilio or MSG91 for SMS delivery (OTP verification, shipping alerts).
 - **Blob Storage:** AWS S3 or Cloudinary for securely hosting user avatars and high-resolution, optimized product imagery delivered via a global CDN.
-

3. Required External Credentials & Prerequisites

To execute this implementation plan without delays, the organization must procure and provision the following third-party credentials, API keys, and cloud environments prior to the commencement of Sprint 1.

3.1 Authentication & Security (Module 1)

- **Google Cloud Console:** OAuth 2.0 Client ID and Client Secret, configured with authorized redirect URIs for both staging and production environments.
- **Facebook Developer Portal:** App ID and App Secret.
- **JWT Cryptography:** Generation of cryptographically secure, high-entropy strings (minimum 256-bit) for signing Access Tokens (short-lived, 15m) and Refresh tokens (long-lived, 7d).

3.2 Payment Gateway (Modules 4 & 6)

- **Primary Payment Provider (Razorpay / Stripe / Cashfree):**

- **Test Mode** API Keys (Publishable Key & Secret Key) strictly for local development and staging environments.
- **Live Mode** API Keys for production deployment.
- Configured Webhook Secrets. This is absolutely crucial for cryptographically verifying asynchronous order confirmation callbacks from the payment provider to prevent spoofed transactions.

3.3 Cloud Storage (Modules 2 & 3)

- **AWS S3 / Cloudinary:**
 - IAM Access Key ID and Secret Access Key with restrictive policies limited only to `s3:PutObject` and `s3:GetObject`.
 - S3 Bucket Name and specific CORS configuration to allow secure, pre-signed multipart uploads directly from the frontend client.

3.4 Communication & Notifications (Module 9)

- **Transactional Email (AWS SES / SendGrid):** SMTP credentials or dedicated API Keys. A verified domain identity with configured DKIM and SPF records to ensure high deliverability (e.g., `noreply@flasherp.com`).
- **SMS Gateway (Twilio / MSG91):** Account SID, Auth Token, and a registered, approved Sender ID compliant with local telecom regulations (for OTPs and delivery updates).

3.5 Infrastructure & Hosting Environments

- **Database:** Provisioned PostgreSQL 15+ instance URI (e.g., AWS RDS, Supabase, Neon) with automated daily backups enabled.
 - **Cache:** Provisioned Redis 7+ cluster URI (e.g., AWS ElastiCache, Upstash) configured with a high-availability architecture and explicit KeySpace event notifications enabled (`notify-keyspace-events Ex`).
-

4. Exhaustive Feature Implementation Plan

The following section provides a granular, deeply technical breakdown of the required modules, their precise business logic, and strict acceptance criteria.

Module 1: Authentication, Security & RBAC (1.5 Weeks)

Scope: Establishing the secure perimeter of the application. This module ensures that only authorized administrators can access internal dashboards and that B2C customers have a seamless, secure onboarding experience.

Detailed Features Breakdown:

- **Admin Authentication Flow:** Secure JWT-based login mechanisms specifically for internal staff. This includes Redis-backed session management to allow instant invalidation of compromised sessions, token refreshing logic, and a rigorous password recovery workflow that sends secure, time-limited reset links to registered admin emails.
- **Customer Authentication Flow:** A frictionless onboarding system for end-users. Features standard registration, secure password login, and OTP (One-Time Password) verification to validate phone numbers or emails and prevent the creation of bot accounts.
- **Social OAuth Integrations:** Integration with Google and Facebook SDKs to allow customers to bypass manual registration. The backend will exchange OAuth codes for user profiles, instantly creating an account and issuing internal JWTs.
- **Role-Based Access Control (RBAC):** NestJS execution context Guards that intercept every incoming HTTP request. These guards decode the JWT, determine the user's role (Admin, Customer, Support), and strictly block unauthorized access to protected routes.

Acceptance Criteria: Admins and customers must exist in distinctly isolated authorization contexts. Passwords must be hashed using bcrypt (salt rounds: 12) before persistence. All public-facing API endpoints must be rate-limited to prevent brute-force attacks.

Module 2: Customer CRM & Account Management (1.5 Weeks)

Scope: Providing customers with complete autonomy over their accounts while giving administrators the tools to comprehensively monitor customer behavior, lifetime value, and order history.

Detailed Features Breakdown:

- **Customer Profile Management:** A self-service REST API portal where customers can update their personal information, change their contact preferences, and upload custom avatars. Avatars will be securely uploaded via pre-signed URLs directly to AWS S3 to offload bandwidth from the API server.
- **Address Book Management:** A robust system allowing customers to save, edit, and soft-delete multiple shipping and billing addresses. Customers can designate "default" addresses to expedite the checkout process during high-pressure flash sales.
- **Admin Customer Directory:** A powerful internal dashboard API for administrators. It features paginated lists, advanced search via Indexed text fields, and filtering. It allows staff to dive into individual customer profiles to audit their lifetime spend, wallet balance, active support tickets, and complete chronological order history.

Acceptance Criteria: Customers can seamlessly switch between multiple addresses during checkout without page reloads. Admin views must render customer data in under 200ms using optimized, paginated database queries.

Module 3: Master Catalog & Inventory Engine (2.0 Weeks)

Scope: The backbone of the e-commerce store. This module governs the creation, categorization, searching, and physical stock management of all products sold on the platform.

Detailed Features Breakdown:

- **Category Administration:** Support for deep, multi-level category nesting (e.g., Electronics > Laptops > Gaming) using Adjacency List or Materialized Path patterns in PostgreSQL. Admins can attach rich promotional banners to specific categories that will render dynamically on the storefront.
- **Product Catalog Management:** Comprehensive CRUD (Create, Read, Update, Delete) controls for the product catalog. This includes managing complex SKUs, standard retail prices (MRP), special Flash Prices, rich-text markdown descriptions, and multi-image galleries.
- **Inventory & Stock Control:** The system for tracking physical warehouse units. Admins can manually inward stock, make real-time adjustments for shrink/damage, and set threshold rules for automated low-stock email alerts.
- **Public Storefront Catalog:** The optimized, read-only engine that serves product listings to the public. Includes advanced filtering (by price range, category, brand), full-text search capabilities, and cursor-based pagination for infinite scrolling.

Acceptance Criteria: The system must strictly prevent the hard deletion of categories or products that are tied to historical orders to maintain referential integrity. Stock levels can never be adjusted below

zero.

Module 4: Flash Deal Engine & Registration Gate (2.5 Weeks)

Scope: The unique selling proposition of the platform. This module controls the scheduling of high-urgency sales and the financial gating mechanism that filters out bots and uncommitted buyers.

Detailed Features Breakdown:

- **Flash Deal Configuration:** Administrative APIs to create and modify Flash Deals. Admins can define the exact UTC start and end timestamps, assign a specific roster of products (with overrides for flash pricing), and determine the required registration fee.
- **Micro-Transaction Registration Gateway:** A specialized checkout flow that occurs *before* the deal starts. Customers must pay a nominal fee (e.g., ₹1) to secure their spot. This payment confirms intent, filters out automated bots, and pre-authenticates their payment method for faster checkout later.
- **Automated Deal State Machine:** Background Node.js Cron jobs that monitor the server clock. Deals automatically transition from *Upcoming* (accepting registrations) to *Live* (accepting purchases) to *Ended* based on strict timestamp comparisons.
- **Eligibility Enforcement:** The security middleware layer that verifies a user's registration status. If a user did not complete the registration fee payment during the *Upcoming* phase, the system will block them from accessing the *Live* deal UI or adding deal items to their cart via the API.

Acceptance Criteria: The state transitions must be perfectly synchronized with the server clock across all distributed instances. Unregistered users must face hard 403 Forbidden blocks if they attempt to participate in a live deal.

Module 5: Atomic Cart & Redis Reservation System (2.0 Weeks)

Scope: The most technically complex and critical module. Designed to completely prevent overselling and inventory race conditions when thousands of users attempt to buy the same limited-stock item simultaneously.

Detailed Features Breakdown:

- **Atomic Stock Deduction (Redis):** When a user clicks "Add to Cart", a single-threaded Redis Lua script executes. It checks available stock and safely decrements it in a single atomic

operation, moving the unit to a temporary "Reserved" state hash mapped to the user's session.

- **10-Minute Cart Time-To-Live (TTL):** Every reservation is stamped with a strict countdown timer. The customer has exactly 10 minutes to complete checkout. This TTL is managed directly by Redis key expiry mechanisms.
- **Automated Stock Release:** If the 10-minute timer expires and the checkout is not completed, a Redis KeySpace event listener (running in the NestJS microservice) instantly captures the expiry payload and automatically increments the stock back into the global available pool for other buyers.
- **Live Stock Broadcasting:** Utilizing WebSockets (Socket.io) to broadcast the exact number of available units to all active viewers in real-time, driving massive urgency without requiring clients to poll the server.

Acceptance Criteria: Absolute guarantee of zero race conditions. If 10 items remain and 1,000 users click "Add to Cart" at the exact same millisecond, exactly 10 transactions must succeed and 990 must be gracefully rejected with a 409 Conflict "Out of Stock" notification.

Module 6: Checkout, Payments & Wallet System (2.0 Weeks)

Scope: The financial core of the platform, handling money capture, tax calculations, and the internal closed-loop wallet ecosystem.

Detailed Features Breakdown:

- **Dynamic Order Calculation:** The engine that computes the final cart subtotal. It dynamically calculates Goods and Services Tax (GST) based on the user's shipping state, applies logic-based shipping rates, and deducts any available wallet balances.
- **Primary Payment Gateway Integration:** The seamless integration with providers like Razorpay or Stripe to capture credit card, UPI, or net banking funds securely. This involves creating order intents and securely verifying payment signatures upon callback.
- **Customer Wallet Ecosystem:** A closed-loop financial system. Customers can manually top-up their wallets in advance. Refunds for returns or failed flash deals are credited directly to this wallet to retain capital within the platform.
- **1-Click Wallet Checkout:** A crucial feature for Flash Deals. If a customer has sufficient wallet balance, they can bypass the slow external payment gateways entirely, paying for the flash deal instantly with a single click.

Acceptance Criteria: Wallet transactions must be strictly logged in a double-entry database ledger to prevent balance discrepancies or race conditions. Tax calculations must be compliant with regional laws.

Module 7: Order Lifecycle, Returns & Support (2.0 Weeks)

Scope: Everything that happens after the customer's money is captured, ensuring smooth fulfillment, logistics tracking, and customer satisfaction.

Detailed Features Breakdown:

- **Order Confirmation Workflows:** Webhook listeners that securely verify payment success signatures. Once verified, the temporary Redis reservation is permanently cleared, and the stock is permanently deducted from the PostgreSQL master database.
- **Fulfillment & Tracking:** Interfaces for warehouse staff to process orders. Admins can update statuses from *Processing* to *Shipped* to *Delivered*, and attach courier tracking links for the customer to view in their portal.
- **Returns Management:** A portal for customers to initiate returns, requiring them to submit standardized reasons and photo evidence. Admins have a dedicated dashboard to approve or reject these requests and issue refunds back to the user's wallet or original payment method.
- **Ticketing & Support System:** An integrated helpdesk where customers can raise queries regarding specific orders, and admins can reply to resolve issues directly within the ERP without needing external support software.

Acceptance Criteria: The system must automatically generate compliant PDF Tax Invoices the moment an order is confirmed, saving them to AWS S3 and making them instantly available for customer download.

Module 8: Analytics, Reports & Global Settings (1.5 Weeks)

Scope: Providing critical business intelligence to the merchant and offering controls to manage global platform variables without requiring code redeployments.

Detailed Features Breakdown:

- **Live Flash Monitor:** A mission-control dashboard for admins. During a live flash sale, this dashboard uses WebSockets to display real-time active users, current cart holds, and incoming revenue second-by-second.
- **Historical Business Metrics:** Aggregated analytics providing deep insights into total historical revenue, top-selling SKUs, cart abandonment rates, and customer acquisition growth over time.
- **Data Export & Reporting:** Tools to generate and download comprehensive CSV and PDF reports for accounting audits, tax filing, and inventory reconciliation.

- **Global System Configuration:** Admin panels to dynamically control overarching platform variables stored in a dedicated database table, such as the official Platform Name, the global Support Email, and the default Registration Fee amount for new deals.

Acceptance Criteria: The Live Monitor must function flawlessly under heavy load, updating instantly without ever requiring the admin to manually refresh the page. Database aggregation queries must be highly optimized using appropriate indices.

Module 9: Notifications, Newsletters & Wishlist (1.0 Week)

Scope: Engagement tools designed to retain customers, recover abandoned carts, and explicitly remind them of upcoming sales they have registered for.

Detailed Features Breakdown:

- **Automated Alert System:** The infrastructure for sending push notifications, emails, and internal app alerts. Crucially, this system uses a background queue (BullMQ/Redis) to reliably dispatch automated SMS/Email reminders exactly 15 minutes before a deal (that the user registered for) goes live.
- **Marketing Newsletter Integrations:** Capture systems across the storefront that allow users to subscribe to email marketing lists, with potential integrations to Mailchimp or Klaviyo.
- **Wishlist Functionality:** Allowing users to save products they are interested in. This provides the business with valuable data on product demand before scheduling future flash deals.

Acceptance Criteria: Alerts and reminders must be dispatched reliably and on-time, utilizing robust queueing mechanisms (like BullMQ) to prevent delays or dropped messages during high-volume send-outs.

Module 10: Complete Frontend Integration (2.0 Weeks)

Scope: The final synthesis phase, wiring the existing `client/` React codebase to the newly developed, fully functional `server/` backend architecture.

Detailed Features Breakdown:

- **API Client Implementation:** Configuring secure Axios interceptors across the React application to handle JWT injection on every request, silent token refreshing, and global unauthorized request redirects to the login page.

- **Dynamic Data Hydration:** Systematically removing all static mock data arrays (e.g., `dummy-data.ts`) and replacing them with live data-fetching hooks using React Query to ensure data is always fresh.
- **UX Polish & Latency Handling:** Implementing comprehensive React error boundaries, skeleton loaders, and localized loading spinners to ensure the UI handles network latency, 500 errors, and API timeouts gracefully without breaking the user experience.

Acceptance Criteria: The entire end-to-end flow—from User Registration to Deal Registration, Cart Reservation, and Final Checkout—must execute seamlessly on the live UI without a single console error or unhandled promise rejection.

5. Quality Assurance & Load Testing Strategy

Given the strict transactional requirements of the Flash Deals engine, a rigorous, multi-layered testing strategy is mandatory before production deployment.

5.1 Unit & Integration Testing

- **Backend (NestJS/Jest):** Comprehensive unit tests covering all critical business logic, specifically the JWT generation/validation, pricing calculation engine, and RBAC middleware. Integration tests will be written using Supertest to validate API endpoints against a localized test database.
- **Frontend (React Testing Library):** Testing critical UI components, particularly the Flash Deal registration gate and checkout flow, ensuring loading states and error boundaries render correctly under failure conditions.

5.2 End-to-End (E2E) Testing

- **Framework:** Cypress or Playwright.
- **Scope:** Automated scripts that simulate the complete critical path: A user registering for an account -> verifying OTP -> registering for a flash deal -> adding an item to the cart -> paying via wallet -> and viewing the generated invoice.

5.3 Stress & Concurrency Testing (The "Hug of Death")

- **Framework:** Artillery or K6.

- **Objective:** Proving the resilience of the Redis Atomic Cart system.
 - **Execution:** We will simulate 10,000 concurrent Virtual Users (VUs) attempting to execute the `POST /cart/add` endpoint simultaneously for a product that only has 50 units in stock.
 - **Acceptance:** The test must prove that exactly 50 requests succeed (Status 200) and exactly 9,950 requests fail (Status 409 Conflict), with zero database locking timeouts or server crashes.
-

6. Deployment & DevOps Strategy (CI/CD)

To ensure rapid, safe, and zero-downtime releases, the infrastructure will heavily rely on automated DevOps pipelines.

- **Containerization:** Both the Next.js frontend and NestJS backend will be fully Dockerized. Multi-stage Docker builds will be utilized to keep the production images lightweight and secure.
- **CI/CD Pipelines:** GitHub Actions (or GitLab CI) will enforce automated workflows. Every Pull Request will trigger a pipeline that runs the Jest unit tests, Prettier formatting checks, and ESLint static analysis. Code cannot be merged unless the pipeline is green.
- **Environment Parity:** We will maintain strict isolation between `Staging` (for internal UAT) and `Production`. Databases and Redis instances will be entirely distinct.
- **Zero-Downtime Deployment:** Utilizing a Blue-Green deployment strategy via AWS ECS or managed container platforms. This guarantees that if a hotfix must be deployed during an active Flash Deal, the traffic is safely routed to the new containers without dropping a single active websocket connection or HTTP request.

7. Monitoring, Logging & Observability

If the system experiences degradation under the massive load of a Flash Deal, the engineering team must be alerted before the customers notice.

- **Centralized Logging:** The NestJS backend will use structured JSON logging (via Pino or Winston). These logs will be shipped to a centralized aggregator (e.g., Datadog, AWS CloudWatch, or ELK Stack) allowing developers to trace request IDs across microservices.
- **Application Performance Monitoring (APM):** An APM tool (e.g., New Relic, Datadog) will be attached to the Node.js process to monitor CPU spikes, memory leaks, and most importantly, identify slow PostgreSQL query bottlenecks.

- **Error Tracking:** Sentry will be integrated into both the React frontend and NestJS backend. Any unhandled exceptions will immediately generate an incident ticket with the exact stack trace and user context.
- **Automated Alerting:** PagerDuty or Slack Webhooks will be configured. If server CPU exceeds 80%, or if the Redis connection drops for more than 5 seconds, an immediate high-priority alert will page the on-call engineer.

8. Security & Compliance

Beyond basic Authentication, the platform must protect both its infrastructure from malicious actors and its data from breaches.

- **Web Application Firewall (WAF):** Placed in front of the application (e.g., Cloudflare, AWS WAF) to automatically detect and block Distributed Denial of Service (DDoS) attacks, which are common during high-profile flash sales.
- **Strict API Rate Limiting:** Global rate limiting will be enforced via Redis to prevent bot networks from brute-forcing login endpoints or scraping the product catalog.
- **Data Privacy & Encryption:** All Personally Identifiable Information (PII) such as customer names, phone numbers, and addresses will be encrypted at rest in the PostgreSQL database using AES-256 encryption. This ensures compliance with modern data protection regulations (such as GDPR or the DPDP Act).
- **PCI-DSS Compliance Offloading:** The FlashERP servers will *never* touch or store raw credit card numbers. All sensitive financial data is tokenized and handled entirely by the payment gateway's secure iframe elements.

9. Master Project Timeline & Milestones (19 Weeks)

The project will be executed in a structured, agile approach divided into seven distinct phases.

Phase	Weeks	Focus Area	Key Deliverables & Milestones
Phase 1: Foundation	1 - 3	Database Schema, Auth (Mod 1), CRM (Mod 2)	PostgreSQL ERD finalized. NestJS boilerplate configured. Users and Admins can securely log in and manage profiles.

Phase 2: Core Data	4 - 5	Catalog (Mod 3), Global Settings	Admins can create, update, and manage categories and complex products with image uploads.
Phase 3: The Engine	6 - 9	Flash Deals (Mod 4), Redis Cart (Mod 5)	Deal scheduling logic active. Redis Lua scripts deployed for atomic stock locking. 10-min cart countdowns functioning perfectly.
Phase 4: Revenue	10 - 13	Checkout, Wallets (Mod 6), Orders (Mod 7)	Payment gateway integration complete. Wallets functional. Automated invoice generation deployed.
Phase 5: Operations	14 - 16	Analytics (Mod 8), Notifications (Mod 9)	Live admin dashboards functional via WebSockets. SMS/Email asynchronous queue pipelines active.
Phase 6: Integration	17 - 18	Frontend Hookup (Mod 10)	UI is fully dynamic and hooked to the API.
Phase 7: QA & Launch	19	Testing (Sec 5) & Deployment	Artillery Load testing complete. E2E pipelines green. Launch ready.

10. Technical Risks & Strict Mitigation Strategies

Given the high-stakes nature of Flash Deals, the architecture must proactively mitigate the following critical risks.

1. Critical Risk: Inventory Overselling during High-Traffic Flash Deals.

- **Mitigation:** Strict, absolute reliance on single-threaded Redis Lua scripts for all initial stock deductions. The PostgreSQL database will only be updated asynchronously via a message queue after the order is fully confirmed and paid for. This prevents database row locking bottlenecks that cause timeouts.

2. Critical Risk: Unacceptable API Latency Under Load (The "Hug of Death").

- **Mitigation:** Aggressively cache public product catalogs and active deal listings using Redis to prevent database hits for read operations. Offload all image delivery and static asset hosting to a dedicated Content Delivery Network (CDN) such as AWS CloudFront.

3. Critical Risk: Unpaid Cart Hoarding Depleting Available Stock.

- **Mitigation:** Implement rigorous Redis KeySpace event listeners within a dedicated microservice. The exact millisecond a 10-minute cart reservation expires in Redis, the event listener must capture it and automatically return the locked stock to the available pool, preventing malicious actors from holding stock hostage.

End of Document. Awaiting Team Lead and Management review.